

Rudy Drive

Gabriel Asenjo¹ Pablo Cisterna² Martin Maza³ Rudy Richter⁴

Universidad Austral de Chile, Valdivia, Chile

Resumen Este trabajo propone el diseño e implementación de un sistema de almacenamiento distribuido basado en erasure coding, utilizando tecnologías open-source como MinIO, FastAPI y RabbitMQ. La solución aborda las limitaciones de los sistemas centralizados en términos de disponibilidad, escalabilidad y tolerancia a fallos, eliminando el punto único de falla mediante fragmentación y redundancia de datos. La arquitectura integra comunicación síncrona y asíncrona, junto con procesamiento distribuido sobre un clúster de bajo costo.

Keywords: Sistema Distribuido · Almacenamiento · Escalabilidad

1. Introducción

El gran problema de los sistemas centralizados son sus limitaciones en términos de disponibilidad y escalabilidad. Al concentrar todo en un solo nodo, se crea un punto único de falla (*SPOF*), lo que centraliza la estabilidad de todo el servicio a la salud de esa única máquina. En este escenario, cualquier error de hardware o corte de red no es solo un incidente técnico, sino un riesgo crítico de pérdida de datos y caída del servicio.

Al comparar con los modelos de fallas en sistemas distribuidos, los sistemas centralizados son mucho mas vulnerables a fallas por detención y fallas por omisión, por su notable falta de mecanismos de redundancia, enmascaramiento de fallas y recuperación automática. En cambio, los sistemas distribuidos asumen la ocurrencia de fallos como un comportamiento natural del sistema por lo que se emplean técnicas como replicación y particionamiento para garantizar cierta confiabilidad del sistema.

Los sistemas de almacenamiento centralizados presentan limitaciones estructurales críticas en disponibilidad, escalabilidad y tolerancia a fallos, esto es debido a la existencia de un single point of failure (SPOF). El crecimiento del volumen de datos y la necesidad de concurrencia impulsan el uso de arquitecturas distribuidas, las cuales emplean replicación, particionamiento y comunicación distribuida para mejorar la confiabilidad. En este contexto, se propone un sistema de almacenamiento distribuido basado en **erasure coding** implementado a nivel de software, utilizando MinIO como sistema de almacenamiento de objetos.

Sistemas como el **Google File System** (GFS) han demostrado la efectividad de arquitecturas distribuidas con buena tolerancia a fallos de hardware estándar, usando técnicas como la replicación de datos, chunking y recuperación automática al sufrir fallos. Sin embargo, estas soluciones no son abiertas al publico, porque Google limita el acceso a sus implementaciones reales, lo cual dificulta su replicación en otros entornos. Como alternativa, estan en auge sistemas open-source que están altamente inspirados por estos diseños, como **Hadoop Distributed File System** (HDFS), que permite experimentar con arquitecturas distribuidas a un menor costo.

En este contexto, se propone el diseño de un sistema de almacenamiento distribuido similar al **cloud storage**, usando una arquitectura de **erasure coding** implementado en software distribuido. Esta arquitectura permite dividir archivos en bloques (**chunking**) y distribuirlos entre muchos nodos junto con sus bloques de paridad, entregando una tolerancia a fallos mediante enmascaramiento. Si falla un nodo, los datos se pueden reconstruir dinámicamente desde la información redundante, siguiendo los principios establecidos en arquitecturas de **erasure coding**.

La contribución principal de este trabajo es establecer una arquitectura distribuida eficiente y efectiva, basada en tecnologías open-source, que logre integrar comunicación asíncrona, buena tolerancia a fallos y

escalabilidad horizontal. Este enfoque busca optimizar el balance del rendimiento, disponibilidad y costo, entregando una solución replicable en contextos académicos y de menor presupuesto.

En las siguientes secciones se describen el modelo físico del sistema, los modelos fundamentales de diseño, la justificación tecnológica y la arquitectura de la base de datos.

2. Arquitectura del sistema

2.1. Componentes del Sistema

La solución utiliza un enfoque **híbrido**, combinando software libre con desarrollo propio.

- **Software libre:** FastAPI, PostgreSQL, RabbitMQ, MinIO, Nginx, React, Docker, WireGuard.
- **Desarrollo propio:** API backend y workers de procesamiento.

Justificación: Se selecciona software libre para reducir costos, evitar dependencia de proveedores y facilitar la escalabilidad. El desarrollo propio permite adaptar la lógica a los requerimientos específicos del sistema distribuido. Estas herramientas permiten construir una solución eficiente, segura y compatible con una arquitectura distribuida basada en procesamiento asíncrono y almacenamiento distribuido.

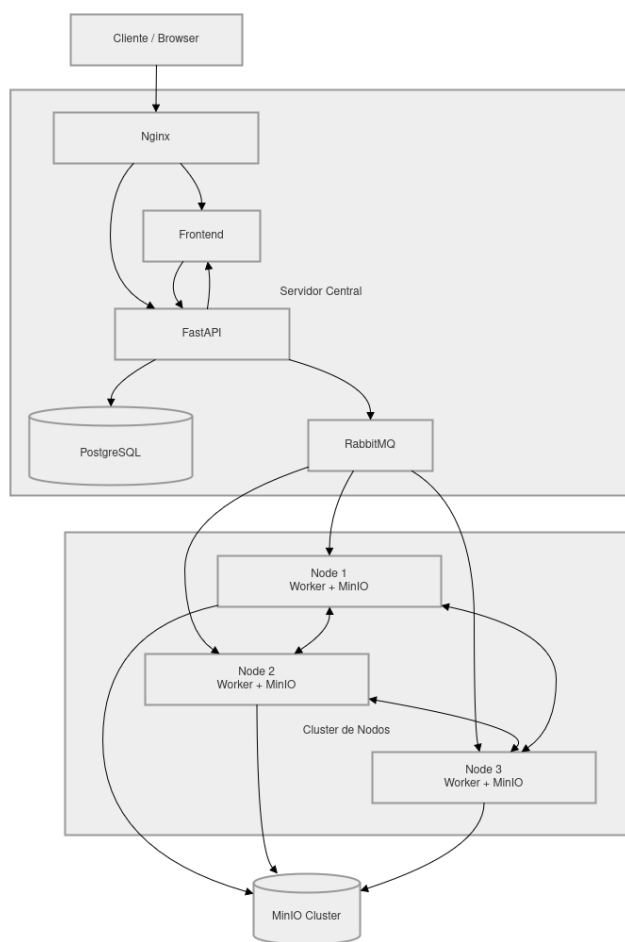


Figura 1. Diagrama de arquitectura de componentes

- **Frontend (Cliente Web)**
 - **Tecnología:** React.js
 - **Versión:** 18.x
 - **Descripción:** Interfaz de usuario para carga y descarga de archivos.
 - **Justificación:** Permite desacoplar la interfaz del backend y mejorar la escalabilidad.
- **Servidor Web / Reverse Proxy**
 - **Tecnología:** Nginx
 - **Versión:** 1.24.x
 - **Descripción:** Punto de entrada al sistema y balanceador de carga.
 - **Justificación:** Mejora la distribución de tráfico y centraliza el acceso.
- **API Gateway**
 - **Tecnología:** FastAPI
 - **Versión:** 0.110.x
 - **Descripción:** Gestiona autenticación y lógica del sistema.
 - **Justificación:** Alto rendimiento y soporte asíncrono nativo.
- **Base de Datos de Metadatos**
 - **Tecnología:** PostgreSQL
 - **Versión:** 15.x
 - **Descripción:** Almacena metadatos de archivos.
 - **Justificación:** Base de datos robusta y confiable para datos estructurados.
- **Sistema de Mensajería**
 - **Tecnología:** RabbitMQ
 - **Versión:** 3.13.x
 - **Descripción:** Distribuye tareas entre nodos.
 - **Justificación:** Permite balanceo de carga y procesamiento asíncrono.
- **Nodos de Procesamiento**
 - **Tecnología:** Python Workers
 - **Versión:** Python 3.11
 - **Descripción:** Ejecutan tareas y gestionan almacenamiento.
 - **Justificación:** Permiten procesamiento distribuido y escalabilidad horizontal.
- **Sistema de Almacenamiento Distribuido**
 - **Tecnología:** MinIO
 - **Versión:** RELEASE.2024-02-17T00-00-00Z
 - **Descripción:** Almacenamiento distribuido de objetos.
 - **Justificación:** Implementa replicación y tolerancia a fallos automáticamente.
- **Red Privada Segura**
 - **Tecnología:** WireGuard
 - **Versión:** 1.0.x
 - **Descripción:** Comunicación segura entre nodos.
 - **Justificación:** Garantiza seguridad en entornos distribuidos.
- **Contenerización**
 - **Tecnología:** Docker
 - **Versión:** 24.x
 - **Descripción:** Encapsula servicios del sistema.
 - **Justificación:** Permite portabilidad, aislamiento y despliegue reproducible.

2.2. Bibliotecas y Frameworks de Python

- **FastAPI:** Framework para implementar la API REST. Se utiliza por su alto rendimiento y soporte asíncrono.
- **Uvicorn:** Permite ejecutar FastAPI de forma eficiente.
- **SQLAlchemy:** ORM para gestionar la interacción con PostgreSQL.
- **Pika:** Cliente para comunicación con RabbitMQ, permitiendo manejo de colas.
- **MinIO SDK:** Cliente para interactuar con el sistema de almacenamiento distribuido.
- **Pydantic:** Validación y serialización de datos en la API.
- **Python-Jose (JWT):** Manejo de autenticación mediante tokens.
- **Passlib:** Hash seguro de contraseñas.

2.3. Modelo Físico

El sistema se implementará bajo un esquema de **cloud computing**, usando dispositivos físicos (notebooks) interconectados dentro de una red privada a modo de **cluster**. Estará compuesta por un total de **cuatro dispositivos**, organizados de la siguiente manera:

- **Servidor Central:** encargado de ejecutar los componentes principales del sistema, incluyendo el servidor web (Nginx), la API backend (FastAPI), el sistema de mensajería (RabbitMQ), servir el frontend (React) y la base de datos (PostgreSQL). Este dispositivo actúa como punto de entrada y orquestador de la lógica del sistema.
- **Nodos de Almacenamiento (3):** cada uno ejecutará un worker de procesamiento y una instancia de almacenamiento MinIO. Estos nodos son responsables de almacenar los fragmentos de archivos de forma distribuida, permitiendo tolerancia a fallos y escalabilidad horizontal.

Todos los dispositivos correrán el sistema operativo **Ubuntu Server 22.04 LTS** y se encontrarán conectados mediante una VPN **WireGuard**, lo que permite la comunicación entre componentes.

3. Modelos Fundamentales

3.1. Modelo de Interacción

El sistema implementa un modelo híbrido de comunicación:

- **Comunicación síncrona (HTTP/REST):** Utilizada entre el cliente y el gestor central para operaciones de control como la autenticación.
- **Comunicación asíncrona (RabbitMQ):** Utilizada para la transferencia de fragmentos de archivos, lo que reduce tiempos de respuesta percibidos y evita bloqueos.

Se sigue este diseño por la ventaja de tener asincronía, lo cual mejora la escalabilidad y flexibilidad con distintos tamaños de carga.

3.2. Modelo de Fallas

El sistema asume un modelo de fallas basado en:

- **Fallos por detención**
- **Fallos por omisión**

Para mitigar el impacto de estos fallos, el sistema utiliza MinIO en modo distribuido, el cual implementa mecanismos de redundancia mediante erasure coding. Este enfoque permite la fragmentación de los datos en múltiples bloques, junto con información de paridad, distribuidos a través de los distintos nodos en el clúster.

3.3. Modelo de Seguridad

El sistema tendrá protección de interfaces utilizando:

- Tokens JWT para autenticación
- Separación de responsabilidades entre componentes
- Cifrado en tránsito (TLS)
- Cifrado en reposo
- Gestión de claves

4. Modelo de Base de Datos

El sistema utiliza PostgreSQL como gestor relacional, con las siguientes entidades principales: El diccionario de datos detallado se encuentra en un anexo en formato Excel.

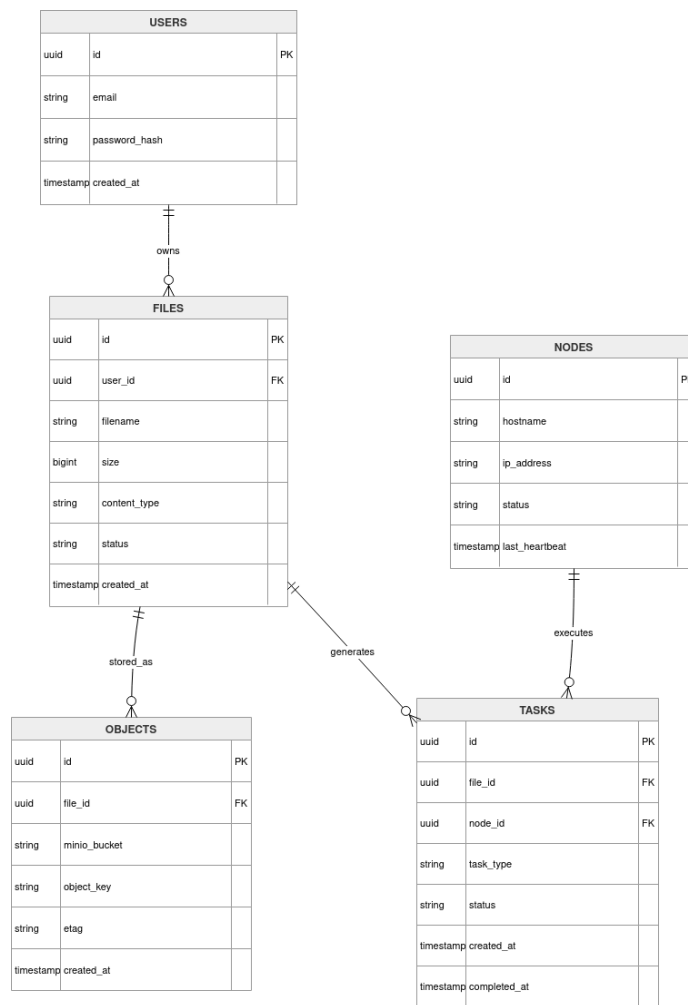


Figura 2. Modelo de base de datos

5. Presupuesto Estimado

Recurso	Cantidad	Costo (CLP)	Total
Laptop Intel i5 8 Nucleos	2	788.490	1.576.980
Laptop Ryzen 5 12 Nucleos	1	799.990	799.990
Laptop Ryzen 3 2 Nucleos	1	348.990	348.990
Total			2.725.960 CLP

Cuadro 1. Estimación de costos

No hay costo mensual debido a que todos los sistemas incluidos son open source o creados por nosotros.

6. Preguntas al Cliente y Supuestos

- ¿Qué sistema operativo utilizan los servidores?**
Los servidores corren Linux bajo la distribución Ubuntu Server 24.02 LTS.
- Con cuantos dispositivos cuentan y cuales son sus características?**
Contamos con 4 dispositivos con:
 - 2 Intel I5 con 8 nucleos, 16GB de RAM y 500GB SSD
 - Ryzen 5 con 12 nucleos, 16GB de RAM y 500GB SSD
 - Ryzen 3 con 2 core, 8GB RAM y 1TB SSD
- Que tipo de archivos se van a almacenar?**
Archivos de propósito general como imágenes (JPG, PNG), documentos (PDF), archivos de texto y documentos de oficina (DOCX, XLSX).
- Cuantos usuarios simultáneos estima que usara el sistema?**
Se estima un rango de entre 20 y 50 usuarios activos simultáneamente.
- El acceso al sistema debe ser publico o por usuarios autenticados?**
El acceso deberá ser de un acceso a usuarios autenticados.
- Que nivel de disponibilidad se requiere para el sistema?**
El sistema debe contar con un SLA del 99 %
- Que tan importante es la perdida de archivos?**
Muy importante, el sistema debe asegurar la disponibilidad continua de los archivos ante fallos.
- Existe un presupuesto para licencias de software?**
Lo ideal es evitar servicios de pago y se usarán herramientas open source
- Se requiere algún control de la cantidad de información que un usuario pueda subir al sistema?**
Cada usuario contara con un limite mensual establecido.
- Cuanto tiempo de vida de los archivos dentro del sistema?**
Los archivos sin uso tendrán un tiempo de vida de un año. Tras eso serán eliminados.